

CS336 第二次作业（系统）：系统与并行性

1.0.4版

2025年春季

1 任务概述

本任务将帮助您掌握提升单GPU训练速度及将训练扩展至多GPU的实践经验。

您将要实施的措施。

1. 基准测试和分析工具
2. Flash Attention 2 Triton内核
3. 分布式数据并行训练
4. 优化器状态分片

代码的具体形式。所有作业代码及本说明文档均可在 GitHub 获取，地址为：github.com/stanford-cs336/assignment2-systems

请通过git克隆该仓库。如有更新，我们将通知您，您可执行`git pull`获取最新版本。

1. `cs336-basics/`: 在本次作业中，您需要分析我们在作业1中构建的部分组件。该文件夹包含作业1的`staf`解决方案代码，因此您会在此找到`cs336-basics/pyproject.toml`和`cs336-basics/cs336_basics/*`模块。若需使用自定义模型实现，可修改根目录下的`pyproject.toml`文件，使其指向您自己的软件包。
2. `/`: `cs336`系统的基础目录。我们创建了一个名为 `cs336_systems` 的空模块。请注意，这里没有代码，因此你可以从头开始自由编写。
3. `tests/*.py`: 该文件包含所有必须通过的测试。这些测试调用`tests/adapters.py`中定义的钩子。您将实现适配器以将代码连接到测试。编写更多测试和/或修改测试代码有助于调试代码，但您的实现应能通过原始提供的测试套件。
4. `readme.md`: 该文件包含有关预期目录结构的更多详细信息，以及一些关于设置环境的基本说明。

如何提交。请将以下文件提交至Gradescope:

- `writup.pdf`: 请回答所有书面问题。请将您的回答排版。
- `code.zip`: 包含您所编写的所有代码。

运行`test_and_make_submission.sh`脚本以创建代码.zip文件。

在作业的第一部分，我们将重点研究如何优化Transformer模型的性能，以实现GPU资源的高效利用。通过性能分析，我们能精准掌握模型在前向和反向传播过程中的时间与内存消耗情况，进而采用定制GPU内核优化自注意力机制，使其运算速度超越PyTorch原生实现。在后续任务中，我们将重点探索多GPU协同工作模式。

1.1 企业形象分析与基准比较

在实施任何优化措施前，建议先对程序进行性能分析，以明确其资源消耗（如运行时间和内存占用）的具体位置。否则，我们可能会优化模型中那些对时间和内存占用影响不大的部分，从而无法获得可量化的整体性能提升。

我们将采用三种性能评估方案：(a)使用Python标准库进行端到端基准测试，记录前向和反向传递耗时；(b)通过 NVIDIA NsightSystems工具分析计算资源分配，了解CPU和GPU操作间的耗时分布；(c)分析内存使用情况。

1.1.1 设置——导入基础变换器模型

首先请确认已成功加载上一作业的模型。在上一作业中，我们已在Python包中搭建了模型框架，以便后续轻松导入。模型的staf实现已添加至./cs336-basics文件夹，并在pyproject.toml文件中进行了配置。按照常规操作调用uv run [command]命令时，uv会自动定位到本地的cs336-basics包。若您需要使用自定义模型实现，可修改pyproject.toml文件，将其指向您自己的包。

您可通过以下方式验证模型导入功能：

```
1 ~$ uv 运行 Python
2 使用CPython 3.12.10
3 在 /path/to/uv/env/dir 创建虚拟环境
4     构建的cs336系统 @ 文件路径/目录/系统/目录
5     构建了cs336-basics @ 文件路径: /path/to/basics/dir
6 已安装85个软件包在711毫秒内
7 Python 3.12.10（主版本，2025年4月9日，04:03:51）[Clang 20.1.0] 在 Linux 上 ...
.
9 >>> 导入 cs336_basics
10 >>>
```

任务1的相关模块现在应该可以使用了（例如，用于模型.py，你可以用import cs336_basics 模型导入它。

1.1.2 模型标定

在整个任务中，我们将对模型进行基准测试和性能分析，以更好地理解其表现。为了了解事物在规模化时的变化情况，我们将使用并参考以下模型配置。所有模型都将采用10,000的词汇量和4的批量大小，并设置不同的上下文长度。本任务（及后续任务）需要大量结果以表格形式呈现。我们强烈建议您通过代码自动化构建表格用于撰写报告，因为在LaTeX或Markdown中格式化表格可能非常繁琐。请参阅pandas.DataFrame.to_latex()和pandas.DataFrame.to_markdown()，或编写自定义函数根据您的表格格式生成表格。

尺寸	d_model	d_ff	num_layers	num_heads
小的	768	3072	12	12
中等的	1024	4096	24	16
大的	1280	5120	36	20
单元格	1600	6400	48	25
2.7B	2560	10240	32	32

表1: 不同型号规格参数

1.1.3 端到端基准测试

我们将开发一个简易的性能评估脚本。由于需要测试模型的多种参数组合（如调整精度、交换神经网络层等），建议通过命令行参数配置脚本，以便后续快速运行。同时**强烈推荐使用Sbatch或Slurm平台的submit工具，对模型规模、上下文长度等基准超参数进行批量扫描，实现快速迭代优化。**

首先，我们通过计时前向和后向传递来对模型进行最简单的性能分析。由于我们仅需测量速度和内存，因此将采用随机权重和数据。

衡量性能是微妙的——一些常见的陷阱会导致我们无法衡量我们想要的。在对GPU代码进行基准测试时，需要注意的是CUDA调用是**异步的**。当你调用CUDA内核时，比如调用`torch.matmul`时，函数调用会将控制权返回给你的代码，而不会等待矩阵乘法完成。通过这种方式，CPU可以继续运行，而GPU则负责计算矩阵乘法。另一方面，这意味着简单测量`torch.matmul`调用返回所需的时间，并不能反映GPU实际执行矩阵乘法所需的时间。在PyTorch中，我们可以调用`torch.cuda.synchronize()`等待所有GPU内核完成，从而更准确地测量CUDA内核的运行时间。基于此，我们来编写基础的性能分析框架。

问题（基准测试脚本）：4分

(a) 编写脚本对模型的前向和反向传播进行基础端到端基准测试。具体而言，该脚本需支持以下功能：

- 给定超参数（如层数），初始化模型。
- 生成随机数据批次。
- 运行 w 个热身步骤（在开始计时之前），然后计时执行 n 个步骤（根据参数，可以是仅向前传递，也可以是向前和向后传递）。对于计时，可以使用Python的`timeit`模块（例如，使用`timeit`函数，或使用`timeit`。默认时间函数（`default_timer()`）可获取系统最高精度的时钟信号，因此作为基准测试的默认时间比普通时间函数（`time()`）更为可靠。
- 在每一步操作后调用`torch.cuda.synchronize()`函数。

可交付成果：一个脚本，用于初始化基础Transformer模型（使用指定超参数）、生成随机数据批次，并执行前向与反向传播。

(b) 对1.1.2节所述模型尺寸的前向和后向传递时间进行计算。使用5个预热步骤，并计算10个测量步骤中时间的平均值和标准差。前向传递需要多长时间？后向传递呢？测量结果中是否出现高变异性，还是标准差较小？

交付物：用1-2句话回答并注明时间。

- (c) 基准测试的一个注意事项是不执行预热步骤。在不执行预热步骤的情况下重复您的分析。这将如何影响您的结果？您认为为什么会发生这种情况？同时尝试使用1或2个预热步骤运行脚本。为何结果仍可能不同？

交付物：2-3句话的回复。

1.1.4 N视系统分析器

端到端基准测试无法显示模型在前向和后向遍历过程中占用的时间和内存资源，因此无法揭示具体的优化机会。若想了解程序在各个组件（如函数）中耗费的时间，我们可以使用 *性能分析器*。执行型性能分析器通过在函数执行开始和结束时插入监控代码，从而在函数级别提供详细的执行统计信息（例如调用次数、平均耗时、累计运行时间等）。

标准Python性能分析工具（如CProfile）无法对 CUDA 内核进行性能分析，因为这些内核在GPU上采用异步执行模式。幸运的是，NVIDIA 提供了一个分析器，我们可以通过CLI `nsys`使用，该工具我们已经为您安装好了。在本任务的这一部分，您将使用`nsys`来分析Transformer模型的运行时。使用`nsys`非常简单：只需在前一节的Python脚本前加上`nsys profile`即可运行。例如，您可以对脚本`benchmark.py`进行性能分析，并将输出写入文件`result.nsys.rep`，命令如下：

```
1 ~$ uv 运行 nsys profile -o 结果 python benchmark.py
```

然后，您可以在本地计算机上使用NVIDIA Nsight Systems桌面应用程序查看该配置文件。

在配置文件的 CUDA API行中选择特定 CUDA API调用（在CPU上），将高亮显示 CUDA 硬件行中所有对应的内核执行（在GPU上）。

我们建议您尝试使用`nsys`配置文件的各种命令行选项，以了解其功能。特别说明，通过`--python-backtrace=cuda`参数可获取每个 CUDA API调用的Python回溯信息，但该操作可能增加系统开销。您还可以通过 NVTX 范围对代码进行注释，这些注释会以区块形式显示在性能分析 NVTX 行中，完整记录所有 CUDA API调用及相关内核执行情况。特别需要注意的是，应使用 NVTX 范围来过滤基准测试脚本中的预热步骤（通过在性能分析 NVTX 行中设置筛选条件）。此外，您可以通过注释实现方式来区分模型前向传播与反向传播所涉及的内核，甚至还能通过以下方式定位自注意力层中不同部分对应的内核：

```
1 ...
2 导入torch.cuda.nvtx作为nvtx
3
4  nvtx.范围（“缩放点积注意力”）
5 def注释缩放点积注意力机制（
6     # Q、K、V、掩码
7 ）
8 ...
9 与nvtx.范围（“计算注意力分数”）：
10     # 计算Q与K之间的注意力分数
11
12 与nvtx.范围（“计算softmax”）
13     # 计算注意力分数的softmax
14
15 与nvtx.范围（“最终乘积”）
16     # 计算输出投影
```

返回

您可通过以下方式将基准测试脚本中的原始实现与注释版本进行替换：

```
cs336_basics.model.scaled_dot_product_attention = 注释 scaled_dot_product_attention
```

最后，您可使用`--pytorch`命令行选项与`nsys`配合，自动为调用PyTorch C++ API的代码添加 NVTX 范围注释。

问题（nsys_profile）：5分

使用`nsys`对表1中描述的每个模型大小和128、256、512和1024上下文长度的前向传递、后向传递和优化步骤进行分析（对于较大的模型，使用某些上下文长度可能会导致内存不足，此时只需在报告中注明即可）。

- (a) 你完成前传动作的总耗时是多少？这与我们之前用Python标准库测得的数据一致吗？

交付物：1-2句话的回复。

- (b) CUDA 内核在前向传播过程中占用的GPU时间最多？在模型单次前向传播过程中，该内核被调用多少次？在执行前向和反向传播时，是否是同一内核占用的运行时间最长？（提示：查看‘统计系统视图’下的‘CUDA GPU内核摘要’，并使用 NVTX 范围进行筛选，以确定模型中各部分对应哪些内核。）

交付物：1-2句话的回复。

- (c) 尽管绝大多数浮点运算（FLOPs）发生在矩阵乘法中，但你会注意到还有其他几个内核占用了相当大的运行时间。在前向传播过程中，除了矩阵乘法之外，还有哪些内核占用了非微不足道的运行 CUDA ？

交付物：1-2句话的回复。

- (d) 使用AdamW优化器执行完整训练步骤（包括前向传播、计算损失、反向传播及最终的优化器步骤，与常规训练流程一致）。相较于仅执行推理（仅前向传播）时，矩阵乘法耗时占比有何变化？其他优化器的性能表现又如何？

交付物：1-2句话的回复。

- (e) 在模型前向传播过程中，比较softmax操作与自注意力层内矩阵乘法操作的运行时间。运行时间的差异与浮点运算次数（FLOPs）的差异相比如何？

交付物：1-2句话的回复。

1.1.5 混合精度

在本任务的当前阶段，我们一直采用FP32精度进行运算——所有模型参数和激活值均使用`torch.float32`数据类型。不过，现代 NVIDIA GPU配备了专门的GPU核心（张量核心），可在较低精度下加速矩阵乘法运算。例如，NVIDIA A100规格说明显示，其FP32最大吞吐量为19.5 TFLOP /秒，而采用FP16（半精度浮点数）或BF16（脑浮点数）时，最大吞吐量显著提升至312 TFLOP /秒。因此，使用低精度数据类型将有助于加快训练和推理速度。

然而，若简单地将模型转换为低精度格式，可能会导致模型精度下降。例如，实际应用中许多梯度值过小，无法用FP16精度表示，若直接用FP16精度训练就会导致梯度值归零。为解决这一问题，FP16训练中通常采用损失缩放技术——通过乘以缩放因子放大梯度值，使其不会归零。此外，FP16的动态范围低于FP32，可能导致数值溢出，表现为NaN损失。虽然采用bfloat16进行完整训练通常更稳定（因为bfloat16与FP32具有相同的动态范围），但与FP32相比仍可能影响模型最终性能。

为充分利用低精度数据类型的加速优势，业界普遍采用混合精度训练方案。在PyTorch框架中，这一机制通过torch自动类型转换管理器实现。具体而言，矩阵乘法等特定运算采用低精度数据类型处理，而需要完整动态范围的运算（如累加和归约操作）则保持原样。例如，以下代码会在前向传播过程中自动识别需要在低精度下执行的操作，并将这些操作转换为指定的数据类型：

```
1 模型: torch.nn.模块= ...# 例如你的Transformer 模型
2 dtype: torch.dtype = ... #例如 torch.float16
3 x: torch.Tensor = ... #输入数据
4
5使用torch.autocast(device= "cuda" , dtype=dtype):
6     y = 模型(x)
```

如前所述，即使被累加的张量本身已被降维，通常仍应保持更高精度的累加。以下练习将有助于建立你对此现象的直观理解。

问题（混合精度累加）：1分

运行以下代码并对结果（准确性）进行评注。

```
s = torch.tensor(0,dtype=torch.float32)
对于我在范围内 (1000):
    s += torch.tensor(0.01,dtype=torch.float32)
print(s)

s = torch.tensor(0,dtype=torch.float16)
对于我在范围内 (1000):
    s += torch.tensor(0.01,dtype=torch.float16)
print(s)

s = torch.tensor(0,dtype=torch.float32)
对于我在范围内 (1000):
    s += torch.tensor(0.01,dtype=torch.float16)
print(s)

s = torch.tensor(0,dtype=torch.float32)
对于我在范围内 (1000):
    x = torch.tensor(0.01,dtype=torch.float16)
    s += x.type(torch.float32)
print(s)
```

交付物：2-3句话的回复。

我们将首先将混合精度应用于一个玩具模型以增强直观理解，随后将其应用于我们的基准测试脚本。

问题（基准测试_混合精度）：2分

(a) 考虑以下模型：

```
1 类ToyModel (nn. 模块):  
2     def __init__(self, in_features: int, out_features: int):  
3         超级().__初始化()  
4         自身.fc1 = nn.线性(输入特征, 10, 偏置=False)  
5         自身.ln = nn.LayerNorm(10)  
6         自身.fc2 = nn.线性(10, 输出特征, 偏置=False)  
7         自身.relu = nn.ReLU()  
8  
9 定义正向(自, x):  
10        x = 自身.relu(自身.fc1(x))  
11        x = 自身.ln(x)  
12        x = 自身.fc2(x)  
13        返回x
```

假设我们正在GPU上训练模型，且模型参数最初为FP32。我们希望使用混合精度自动转换至FP16。以下数据类型是什么：

- 自浇铸工艺中的模型参数
- 第一层前馈层的输出（ToyModel.fc1）
- 层归一化输出（ToyModel.ln）
- 模型预测的logits
- 损失
- 以及模型的梯度？

交付成果：上述各组件的数据类型。

(b) 你应该已经注意到，FP16混合精度自动卷积层在处理层归一化时与前馈层存在差异。层归一化中哪些部分对混合精度敏感？若改用BF16而非FP16，是否仍需对层归一化进行特殊处理？其原因何在？

交付物：2-3句话的回复。

(c) 修改基准测试脚本，可选使用BF16混合精度运行模型。针对1.1.2章节所述各语言模型规模，分别测试混合精度与非混合精度下的前向和反向传播耗时。对比全精度与混合精度的测试结果，并分析模型规模变化时的性能趋势。建议使用空上下文无操作上下文管理器进行测试。

交付物：用2-3句话回答，包括你的计时和评论。

1.1.6 内存分析

到目前为止，我们一直在研究计算性能。现在我们将把注意力转移到内存上，这是语言模型训练和推理中的另一个主要资源。PyTorch还内置了强大的内存分析工具，能够实时追踪内存分配情况。

要使用内存分析工具，可按以下方式修改基准测试脚本：


```

1
2 #基准测试脚本中的热身阶段
3
4 # 开始记录内存历史。
5 torch.cuda.memory._record_memory_history (max_entries=1000000) 。
6
7 # 你希望在基准测试脚本中配置的参数
8
9 # 保存pickle 文件以便PyTorch 在线工具加载。
10 torch.cuda.memory._dump_snapshot ( "memory_snapshotpickle" ) 11
12 # 停止记录历史。
13 torch.cuda.memory._record_memory_history (enabled=None)

```

该操作将生成名为memory_snapshotpickle的文件，您可将其导入以下在线工具：https://pytorch.org/memory_viz。该工具不仅能显示整体内存使用时序图，还能查看每个内存分配的具体信息，包括分配大小及指向源代码的堆栈追踪。使用方法是：先在网页浏览器中打开上述链接，然后将Pickle文件拖拽到页面上即可。

接下来，您将使用PyTorch性能分析器来分析模型的内存使用情况。

问题（内存分析）：4分

分析2.的前传、后传及优化步骤。表1中的7B模型，其上下文长度分别为128、256和512。

- (a) 在您的模型分析脚本中添加一个选项，通过内存分析器运行模型。重用部分现有基础设施（例如启用混合精度、加载特定模型规模等）可能会有所帮助。随后运行脚本，获取2.7B模型在仅执行推理（仅前向传播）或完整训练步骤时的内存使用情况。内存使用时序图呈现如何？能否通过峰值判断当前运行阶段？**交付成果：**使用memory_viz工具生成的2.7B模型“活跃内存时序图”两幅图像：一幅展示前向传播阶段，另一幅呈现完整训练步骤（包含前向传播、反向传播及优化器步骤）的运行过程，并附2-3句响应说明。

- (b) 在前向传播过程中，不同上下文长度的峰值内存使用量是多少？而在完成完整训练步骤时又会呈现怎样的情况？

交付成果：一个表格，每个上下文长度包含两个数字。

- (c) 在混合精度模式下，分别计算2.7B模型在前向传播和完整优化步骤中的峰值内存使用量。混合精度是否显著影响内存使用？**交付物：**2-3句话的回复。

- (d) 以2.7B模型为例，在我们的参考超参数下，Transformer残差流中单精度激活张量的大小是多少？请以MB为单位给出该大小（即用字节数除以10242）。

交付物：用1-2句话回答并说明推导过程。

- (e) 现在仔细观察pytorch.org/memory_viz网站上的“Active Memory Timeline”，该图展示了2.7B模型进行前向传播时的内存快照。当你将“Detail”级别调低时，该工具会隐藏对应级别的最小内存分配（例如，将“Detail”设置为10%时，仅显示

（占比10%的最大分配）这些最大分配的具体数值是多少？通过查看堆栈跟踪，你能判断出这些分配来自哪里吗？

交付物：1-2句话的回复。

1.2 基于FlashAttention-2的注意力优化

1.2.1 PyTorch注意力机制的基准测试

您的分析结果表明，注意力层在内存和计算资源方面存在优化空间。从整体结构来看，注意力机制的运作流程包含三个步骤：首先进行矩阵乘法运算，接着执行softmax激活函数，最后再进行一次矩阵乘法运算。

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\text{mask} \left(\frac{Q^T K}{\sqrt{d_k}} \right) \right) V \quad (1)$$

原始注意力机制的实现需要为每个批次/头元素保存形状为 $\text{seq_len} \times \text{seq_len}$ 的注意力分数矩阵，当序列长度过长时，这种矩阵会变得非常庞大，导致任何涉及长输入或输出的任务都可能出现内存不足的错误。我们将参照FlashAttention-2论文的思路，采用注意力核机制——通过分块计算注意力值，彻底避免显式存储 $\text{seq_len} \times \text{seq_len}$ 的注意力分数矩阵，从而实现对超长序列的扩展能力。

问题（pytorch_attention）：2分

(a) 在不同尺度上对注意力机制的实现进行基准测试。编写一个脚本，该脚本将：

- (a) 将批量大小固定为8，且不使用多头注意力机制（即移除头维度）。
- (b) 遍历[16, 32, 64, 128]的笛卡尔积以确定头部嵌入维度 d_{model} ，以及[256, 1024, 4096, 8192, 16384]的笛卡尔积以确定序列长度。
- (c) 为适当大小创建随机输入 Q, K, V 。
- (d) 时间100通过输入信号在注意力中进行前向传递。
- (e) 在开始反向传递前测量内存使用量，并计时100次反向传递。
- (f) 务必进行热身，并在每次前向/后向传递后调用 `torch.cuda.synchronize()`。

请报告这些配置下的运行时长（或内存不足错误）。在何种规模下会出现内存不足错误？对于发现的最小配置之一，需计算其注意力机制的内存使用量（可采用作业1中Transformer模型的内存使用公式）。为反向操作保存的内存如何随序列长度变化？您将采取何种措施消除这种内存开销？

交付成果：一份包含时间安排、内存使用计算结果及1-2段回复的表格。

1.3 JIT编译的注意机制的基准测试

自2.0版本起，PyTorch还内置了强大的即时编译器，能够自动对PyTorch函数进行多项优化（具体实现可参考https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html）。该编译器通过动态分析计算图，可自动生成融合的Triton内核。其操作界面简洁直观，例如若需应用于模型的单层结构，只需执行以下操作：

```

1 layer = SomePyTorchModule (...)
2 compiled_layer = torch.compile(layer)

```

目前，`compiled_layer`在功能上与`layer`完全相同（例如具有前向和反向传播特性）。我们还可以使用`torch`工具对整个PyTorch模型进行编译，包括`compile(model)`命令，甚至可以编译调用PyTorch操作的Python函数。

问题（torch.compile）：2分

- (a) 将注意力基准测试脚本扩展至包含PyTorch注意力机制的编译版本，并将其性能与上述`pytorch_attention`问题中相同配置的未编译版本进行对比。

交付成果：一张表格，对比你编译后的注意力模块与上述`pytorch_attention`问题中未编译版本的前向和后向传递时间。

- (b) 现在，请在端到端基准测试脚本中编译完整的Transformer模型。前向传播的性能会如何变化？而前向传播、反向传播与优化器步骤的组合效果又如何？

可交付成果：一份对比您原生Transformer模型与编译后Transformer模型的表格。

根据我们观察到的序列长度缩放特性，当前算法在处理长序列时仍存在明显不足。即便使用`torch`编译器，现有实施方案在长序列场景下仍存在严重的内存访问模式缺陷。为此，我们将基于Triton框架开发FlashAttention-2的实施方案，通过优化内存访问策略和计算时机，显著提升算法性能。

1.3.1 示例 - 权重和

为帮助您了解Triton及其与PyTorch的交互方式，我们将通过一个“加权求和”操作的示例内核进行讲解。如需获取更多快速掌握Triton的资源，请参阅Triton教程。需要说明的是，这些教程未采用我们将在下文详细讲解的新型便捷块指针抽象机制。

给定一个输入矩阵 X ，我们将它的元素乘以一个列向量 w ，然后对每一行求和，得到矩阵 X 与 w 的矩阵-向量乘积。我们将首先完成这个操作的前向传播，然后编写Triton内核来实现后向传播。

前向传递我们的核函数的前向传递就是以下广播的内积。

```

1 def加权求和(x, weight):
2     此处假设x具有n维形状[... , D]，权重具有l维形状[D]
3     return (weight * x).sum(axis=-1)

```

在编写Triton内核时，每个程序实例（可能并行运行）将计算数据块中 x 行的加权和，并将对应的标量输出写入输出张量。在Triton架构中，程序实例是运行相同程序的线程块集合，这些线程块可在GPU上并行执行。我们不直接接受张量作为参数，而是接收指向其首元素的指针，以及每个张量的步长——这些参数用于指导沿轴向的移动方向。

我们可以使用步长来加载一个与我们正在运行实例中求和的 x 行的瓦片对应的张量，使用程序ID来划分工作（即实例 i 将处理 x 的第 i 行瓦片）。在这一简单案例中，Triton与PyTorch前向传播的主要区别在于需要进行指针运算及显式加载/存储操作。我们将采用块指针抽象机制。

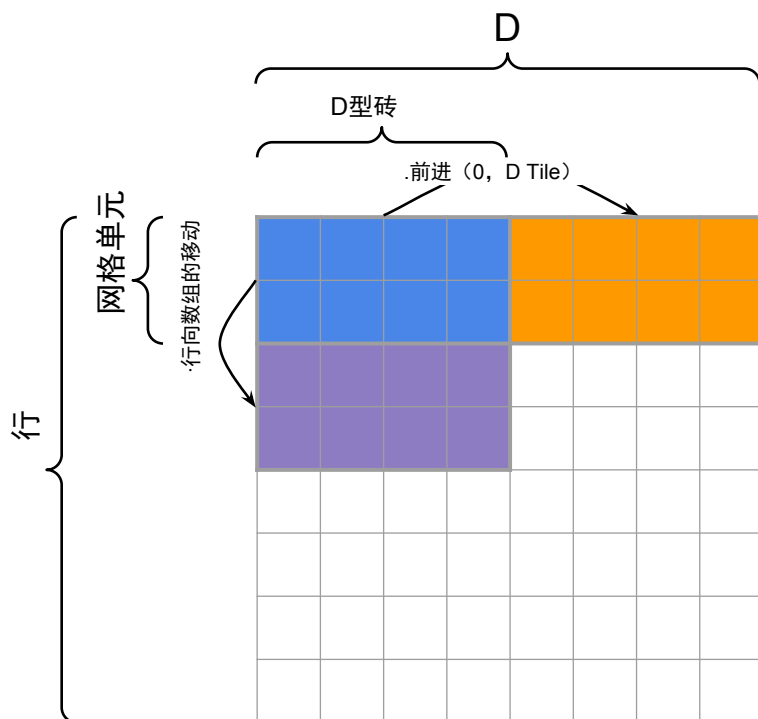


图1：加权和核示例中块指针的平铺与推进（第1.3.1节）。

`tl.make_block_ptr` 旨在极大简化指针运算，但需进行相关设置以准备块指针。

有关平铺示意图及块指针如何推进的说明，请参见图1。上述加权求和函数如下所示：

```

1 导入Triton
2 导入triton .language作为tl
3
4  @triton .吉特
5 def加权求和前向 (
6     x_ptr, weight_ptr, # 输入指针
7     output_ptr, # 输出指针
8     x_stride_row, x_stride_dim, # 步长决定了如何在张量的每个轴上移动一个元素
9     weight_stride_dim, # 可能是 1
10    输出行数, # 可能为 1
11    罗兹, D,
12    ROWS TILE_SIZE: tl.constexpr, D TILE_SIZE: tl.constexpr, # 瓷片形状必须在编译时已知 13):
13    每个实例将计算x的行组成的瓦片的加权和。
14    # `tl.program_id` 可用于确定当前运行的线程块
15
16 行_列索引 = tl.程序ID (0)
17
18    块指针提供了一种从非数据区域 (ND) 内存中进行选择的方法
19    # 并移动我们的选择范围。
20    # 块指针必须知晓:
21    #- 张量首元素的指针
22    #- 处理越界访问的张量整体结构

```

```

23     #- 各维度的步进值以正确使用内存布局
24     #- 起始区块的ND坐标, 即 “偏移量”
25     #- 一次加载/存储时使用的块形状
26     #- 内存中维度的顺序 (从主要到次要)
27     #- 轴 (= np.argsort (步长)) 用于优化, 尤其适用于H100 28
28     x_block_ptr = tl.make_block_ptr(
29         x_ptr,
30         形状=(行数, 数),
31         步长=(x_row_stride, x_stride_dim), 33     偏移量=(行_
32     块索引*行_块大小, 0), 34     块形状=(行数_块大小, 列数_块
33     大小)
34     顺序=(1, 0), 36)
35
36
37
38     weight_block_ptr = tl.make_block_ptr(
39         权重指针
40         形状=(D,
41         步幅=(权重步幅维度),
42         偏移量=(0, ),
43         块形状=(D Tile尺寸, )
44     顺序=(0, ), 45)
45
46
47     output_block_ptr = tl.make_block_ptr(
48         输出指针
49         形状=(行数),
50         步长=(输出步长行),
51         偏移量=(行瓦片索引*行瓦片大小),
52         块形状=(行数×列数),
53     顺序=(0, ), 54)
54
55
56     # 初始化写入缓冲区
57 输出 = tl.zeros ((ROWS TILE_SIZE, ), dtype=tl.float32)
58
59 对于i在范围内 (tl.cdiv (D, D_tile_SIZE)) :
60     # 加载当前块指针
61     由于ROWS TILE_SIZE 可能无法整除ROWS, 且D TILE_SIZE 可能无法整除D,
62     # 我们需要对两个维度进行边界检查
63     row = tl.load (x_block_ptr, boundary_check=(0, 1), padding_option= "zero" ) # (ROWS TILE_SIZE, D TILE_SIZE)
64     weight = tl.load (weight_block_ptr, boundary_check=(0, ), padding_option= "zero" ) # (D TILE_SIZE, ) 65
65     计算行的加权和。
66     output += tl.sum (row * weight[无, :], axis=1)
67
68
69     # 将指针移至下一个方块。
70     # 这些是 (行、列) 坐标增量
71     x_block_ptr = x_block_ptr + (0, D_tile_SIZE) # 在最后一个维度上移动D_tile_SIZE
72     weight_block_ptr = weight_block_ptr + (D_TILE_SIZE, ) # 移动D_TILE_SIZE 个单位 73
73
74     将输出写入输出块指针 (每行一个标量)。
75     由于ROWS TILE_SIZE 可能无法整除ROWS, 因此需要进行边界检查
76 吨. 存储 (输出块指针, 输出, 边界检查=(0, ))

```

现在我们将这个内核封装到PyTorch的Autograd函数中, 使其能够与PyTorch无缝对接 (即接收Tensor作为输入、输出Tensor结果, 并在反向传播过程中与Autograd引擎协同工作):

1 类加权和函数 (torch.autograd. 函数):

2 @静默

3 定义前向 (上下文, x, 权重):

```
4         # 缓存x和权重以供反向传播时使用
5         仅接收输出张量的梯度
```

```

6      # 需要计算关于 x 的梯度和权重。
7      D, 输出维度 = x 的形状[-1], x 的形状[-1]
8
9      # 将输入张量重塑为二维
10     输入形状 = x.形状
11     x = rearrange (x, "...d-> (...) d" )
12
13     ctx.save_for_backward (x, weight)
14
15     断言 len (weight.shape) == 1 且 weight.shape[0]=D, "维度不匹配"
16     断言 x.是CUDA 且 权重.是CUDA, "预期 CUDA 张量"
17     断言 x.是连续的 (), "我们的指针算术将假设x是连续的"
18
19     ctx.D_tile_SIZE=triton.next_power_of_2 (D)// 16# 大约16次遍历嵌入维度
20     ctx.ROWS_TILE_SIZE= 16# 每个线程一次处理16个批次元素
21     ctx .input_shape = 输入形状
22
23     # 需要初始化空结果张量。请注意, 这些元素不一定是0!
24     y = torch.empty (output_dims, device=x .device)
25
26     在我们的1 维网格中以n 个实例启动内核。
27     n 行 = y 数字函数 ()
28     加权前向累加[ (n_rows除以ctx ROWS_TILE_SIZE的商) ]
29         x, 重量,
30         y,
31         x .stride (0) , x .stride (1) ,
32         重量.步幅 (0) ,
33         y .stride(0),
34         ROWS=行数, D=日期,
35         ROWS_TILE_SIZE=ctx . ROWS_TILE_SIZE, D_TILE_SIZE=ctx .D_TILE_SIZE, 36
36     )
37
38     返回 y.view (输入形状[-1])

```

请注意, 当我们调用带有加权求和函数 `weighted_sum_fwd[ (cdiv (n_rows, ctx . ROWS_TILE_SIZE) , ) ]` 的Triton内核时, 通过传递元组 `(cdiv (n_rows, ctx . ROWS_TILE_SIZE) )` 定义了一个所谓的“启动网格”线程块。随后, 我们可以在内核中通过 `tl.program_id (0)` 访问线程块索引。

向后传递 由于我们正在定义自己的内核, 因此还需要编写相应的后向函数。

在前向传播中, 我们获得了层的输入, 并需要计算其输出。在反向传播中, 需要记住我们将获得目标函数关于输出的梯度, 并需要计算关于每个输入的梯度。在我们的案例中, 操作的输入是一个矩阵 $x: \mathbf{R}^{n \times h}$ 和一个权重向量 $w: \mathbf{R}^h$ 。简言之, 我们称该操作为 $f(x, w)$, 其值域为 $\mathbf{R}^n$ 。假设给定 $\nabla_{f(x, w)} L$, 即损失函数 L 关于层输出的梯度, 我们可以应用多元链式法则, 得到关于 x 和 w 的梯度表达式如下:

$$(\nabla_x \mathcal{L})_{ij} = \sum_{k=1}^n \frac{\partial f(x, w)_k}{\partial x_{ij}} (\nabla_{f(x, w)} \mathcal{L})_k = w_j \cdot (\nabla_{f(x, w)} \mathcal{L})_i \quad (2)$$

$$(\nabla_w \mathcal{L})_j = \sum_{i=1}^n \frac{\partial f(x, w)_i}{\partial w_j} (\nabla_{f(x, w)} \mathcal{L})_i = \sum_{i=1}^n x_{ij} \cdot (\nabla_{f(x, w)} \mathcal{L})_i \quad (3)$$

由此可推导出计算反向传递的简易公式。为了获得关于 x 的反向步，我们应用公式2并取 w 与 $\nabla_{f(x, w)}$ 的外积。要计算关于 w 的反向步（即 $(\nabla_w L)_j$ ），我们必须将输入梯度乘以对应的输出行。

我们的反向传播核心将首先定义所有块指针，然后计算 $\nabla_x L$ ：


```

1  @triton.吉特
2 def加权求和反向(
3      x_ptr、weight_ptr、#输入
4      grad_output_ptr, #比率输入
5      grad_x_ptr、partial_grad_weight_ptr、#梯度输出
6      步长_xr, 步长_xd,
7      步宽
8      栅格粒度
9      步长_gx, 步长_gx,
10     步长_gwb, 步长_gwd,
11     NUM_ROWS, D,
12     ROWS TILE_SIZE: tl.constexpr, D TILE_SIZE: tl.constexpr, 13):
14 行_列索引 = tl.程序ID (0)
15 n_row_tiles = tl.num_programs (0)
16
17     # 输入
18     grad_output_block_ptr = tl.make_block_ptr(
19         输出指针,
20         形状=(行数), 步长=(步长),
21         偏移量=(行瓦片索引*行瓦片大小),
22         块形状=(行数*列数),
23         顺序=(0, ),
24     )
25
26     x_block_ptr = tl.make_block_ptr(
27         x_ptr, 28     形状=(行数, D), 步长=(步长_xr, 步长_xd), 29
        偏移量=(行_块索引*行_块大小, 0), 30     块形状=(行数_块大小, 列数_
        块大小)
31         顺序=(1, 0),
32     )
33
34     weight_block_ptr = tl.make_block_ptr(
35         权重指针
36         形状=(D), 步长=(stride_wd),
37         偏移量=(0, ), 块形状=(D_tile_SIZE, )
38         顺序=(0, ),
39     )
40
41     grad_x_block_ptr = tl.make_block_ptr(
42         grad_x指针,
43         形状=(行数, D), 跨步=(步长_xr, 步长_xd)
44         偏移量=(行_磁贴索引*磁贴大小, 0),
45         块形状=(行数_块大小, 列数_块大小)
46         顺序=(1, 0),
47     )
48
49 部分梯度权重块指针 = tl.创建块指针(
50     偏重权指针,
51     形状=(n行瓷砖数, D), 跨步=(步长_gwb, 步长_gwd)
52     偏移量=(行_块索引, 0),
53     块形状=(1, DTile尺寸),
54     顺序=(1, 0),
55 )
56

```

```

57 对于i在范围内 (tl.cdiv (D, D_tile_SIZE)) :
58     grad_output = tl.load (grad_output_block_ptr, boundary_check= (0, ), padding_option= "zero" ) # (ROWS TILE_SIZE, ) 59
60     # 与x的外积
61     weight = tl.load (weight_block_ptr, boundary_check= (0, ), padding_option= "zero" ) # (D TILE_SIZE, )
62     grad_x_row = grad_output[:, 无]*重量[无, :]
63     tl.存储 (grad_x_block_ptr, grad_x_row, boundary_check= (0, 1) )
64
65     尽可能减少grad_weight结果的行数

```

```

66     row = tl.load (x_block_ptr, boundary_check= (0, 1), padding_option= "zero" ) # (ROWS TILE_SIZE, D TILE_SIZE)
67     grad_weight_row = tl.sum (row * grad_output[:, None], axis=0, keep_dims=True)
68     tl.存储 (partial_grad_weight_block_ptr, grad_weight_row, boundary_check= (1, )) # 维度0永远不在边界外
69     # 将指针沿D方向移动到下一个方块
70
71     x_block_ptr = x_block_ptr先进的 ( (0, D_tile_SIZE) )
72     weight_block_ptr = weight_block_ptr.advance ( (D_TILE_SIZE, ) )
73     partial_grad_weight_block_ptr = partial_grad_weight_block_ptr.advance ( (0, D_tile_SIZE) )
74     grad_x_block_ptr = grad_x_block_ptr先进的 ( (0, D_tile_SIZE) )

```

计算梯度 ∇_x 很简单，我们将结果写入输出张量的相应瓦片。然而，计算 ∇_w 则更具挑战性。每个内核实例负责 x 的一个行瓦片，但我们现在需要对 x 的行进行求和 *across*。我们不会在反向传播中直接进行这个求和，而是假设 `partial_grad_weight_ptr` 包含一个 $n_row_tiles \times H$ 矩阵，其中第一维仅在来自 x 的行瓦片内进行归一化。我们在写入该张量之前先在当前行瓦片内进行归一化。在内核外部，我们使用 `torch.sum` 对 ∇_w 进行归一化，以汇总每个行瓦片的结果¹。自回归部分的最终功能相对简单：

```

1 类加权求和函数 (torch.autograd. 函数):
2     @静默
3 定义前向 (上下文, x, 权重):
4     #... (定义见前文)
5
6     @静默
7 def backward (ctx, grad_out):
8     x, 权重 = ctx.saved_tensors
9     ROWS TILE_SIZE, D TILE_SIZE = ctx.ROWS TILE_SIZE, ctx.D TILE_SIZE # 这些不一定要相同
10    n 行, D = x.shape[1]
11    我们的策略是让每个线程块先写入一个部分缓冲区,
12    然后通过这个缓冲区进行梯度下降以获得最终梯度。
13
14    偏置权重 = torch.empty ( (cddiv (n_rows, ROWS TILE_SIZE), D), device=x.device, dtype=x.dtype)
15    grad_x = torch.empty_like(x)
16
17    加权反向步进[ (n_rows, ROWS TILE_SIZE) 除以]
18        x, 重量,
19        输出级
20        梯度_x, 偏置梯度权重,
21        x.stride (0), x.stride (1),
22        重量.步幅 (0),
23        渐变输出步长 (0)
24        grad_x.stride (0), grad_x.stride (1),
25        partial_grad_weight.stride (0), partial_grad_weight.stride (1),
26        NUM_ROWS=n行, D=D,
27    ROWS TILE_SIZE=ROWS TILE_SIZE, D TILE_SIZE=D TILE_SIZE, 28    )
29    grad_weight = partial_grad_weight.sum (axis=0)
30    返回 grad_x, grad_weight

```

最终，我们现在可以得到一个功能类似于 `torch.nn functional` 中实现的函数：

`f_weightedsum` = 加权求和函数的应用

现在，对两个 PyTorch 张量 x 和 w 调用 `f_weightedsum` 将得到如下所示的张量：

张量 `[90.8563, -93.6815, -80.8884, ..., 103.4840, -21.4634, -24.0192]`, 2)

```
device= 'cuda:0' , grad_fn=<WeightedSumFuncBackward>)
```

注意张量所附带的`grad_fn`——这表明PyTorch在计算图中出现该张量时，已知反向传播中应调用的函数。至此，我们基于Triton的加权求和运算实现完成。

¹当然，我们也可以自行开发对应的内核。

1.3.2 FlashAttention-2前向传递

您将用改进版的Triton实现替代PyTorch的注意力机制，该改进基于FlashAttention-2[Dao, 2023]。FlashAttention-2采用特殊技术以分块方式计算前向传播，既能优化内存访问模式，又避免了将完整注意力矩阵存入全局内存的必要性。

在进入本节之前，我们强烈建议至少阅读原始的FlashAttention论文[Dao等人, 2022]，这将帮助你理解实现FlashAttention高效注意力机制的核心技术：以在线方式跨瓦片计算softmax（该技术由[Milakov和Gimelshein, 2018]提出）。我们还建议查阅He[2022]，以获取更多关于GPU如何实际执行PyTorch代码的直观理解。

理解基础注意力机制中的低效问题。回想一下，注意力机制的前向传播（暂不考虑掩码操作）可表示为：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top / \sqrt{d} \quad (4)$$

$$\mathbf{P}_{ij} = \text{softmax}_j(\mathbf{S})_{ij} \quad (5)$$

$$\mathbf{O} = \mathbf{P}\mathbf{V} \quad (6)$$

标准的反向传球是

$$d\mathbf{V} = \mathbf{P}^\top d\mathbf{O} \quad (7)$$

$$d\mathbf{P} = d\mathbf{O}\mathbf{V}^\top \quad (8)$$

$$d\mathbf{S}_i = d\text{softmax}(\mathbf{dP}_i) = (\text{diag}(\mathbf{P}_i) - \mathbf{P}_i\mathbf{P}_i^\top) d\mathbf{P}_i \quad (9)$$

$$d\mathbf{Q} = d\mathbf{S}\mathbf{K} / \sqrt{d} \quad (10)$$

$$d\mathbf{K} = d\mathbf{S}^\top \mathbf{Q} / \sqrt{d}, \quad (11)$$

正如我们所见，反向传播依赖于前向传播中某些非常大的激活值。例如，在公式（7）中计算 $d\mathbf{V}$ 需要 $\mathbf{P}$ ，即形状（batch_size, n_heads, seq_len, seq_len）的注意力分数——该激活矩阵的大小与序列长度呈二次方关系，这解释了我们在基准测试中遇到的大序列长度注意力机制时出现的内存问题。在标准注意力机制的前向和反向传播过程中，我们在片上SRAM与GPU HBM之间传输 $\mathbf{P}$ 及其他大激活值时会产生显著的内存I/O开销。标准实现中存在多次此类传输：例如，在公式（7）和（9）的计算中，标准反向传播实现都会从HBM读取 $\mathbf{P}$ 。

FlashAttention的核心目标是避免频繁读写注意力矩阵HBM，从而降低输入输出（IO）和峰值内存开销。我们通过三种技术实现这一目标：平铺法、重计算法和算子融合法。

平铺。为避免在HBM之间读写注意力矩阵，我们无需访问完整输入即可计算softmax归一化。具体而言，我们将注意力计算重构为将输入分割为平铺块，并对这些平铺块进行多次遍历，从而逐步完成softmax归一化。

重新计算。我们避免将形状为（batch_size, n_heads, seq_len, seq_len）的大型中间注意力矩阵存储在HBM中。相反，我们会在HBM中保存某些“激活检查点”，然后在反向传播过程中重新计算部分前向传播，以获取计算梯度所需的其他激活值。FlashAttention-2还存储了注意力分数的对数和指数（ L ），该值将被

用于简化反向传播计算。 L 的表达式为：

$$L_i = \log \left(\sum_j \exp(\mathbf{S}_{ij}) \right) \quad (12)$$

在我们的最终内核中，我们将以在线方式计算这个值，但最终结果应该是一样的。通过平铺和重新计算，我们的内存IO和峰值使用不再依赖于`sequence_length`²，因此我们可以使用更大的序列长度。

算子融合。最后，我们通过将所有操作集中于单一内核中来避免注意力矩阵及其他中间激活值的重复内存I/O操作——这种技术称为算子或内核融合。我们将为前向传播设计一个单一的Triton内核，该内核执行所有注意力相关操作，同时将 HBM 与 SRAM 之间的数据传输量降至最低。算子融合部分得益于重新计算机制，因为我们可以避免将每个中间激活值存储到 HBM 时通常需要进行的内存I/O操作。

要对这些技术有更直观的理解，请查看FlashAttention论文[Dao等人，2022，Dao，2023]。

倒序传递与重新计算。使用 L ，我们可以进行适当的重新计算并高效地完成反向传播。在开始反向传播之前，我们预先将值 $D = \text{rowsum}(\mathbf{O} \circ \mathbf{dO})$ （其中 $\circ$ 表示逐元素乘法）预计算到全局内存中，该值等于 $\text{rowsum}(\mathbf{P} \circ \mathbf{dP})$ ，因为 $\mathbf{P} \mathbf{dP}^\top = \mathbf{P}(\mathbf{dO} \mathbf{V}^\top)^\top = (\mathbf{P} \mathbf{V}) \mathbf{dO}^\top = \mathbf{O} \mathbf{dO}^\top$ （且对于任意矩阵 $\mathbf{A}$ 和 $\mathbf{B}$ ， $\text{rowsum}(\mathbf{A} \circ \mathbf{B}) = \text{diag}(\mathbf{A} \mathbf{B}^\top)$ ）。通过使用 L 和 D 向量，反向传播计算可无需 softmax。反向传播的完整计算过程如下：

$$\mathbf{S} = \mathbf{Q} \mathbf{K}^\top / \sqrt{d} \quad (13)$$

$$\mathbf{P}_{ij} = \exp(\mathbf{S}_{ij} - L_i) \quad (14)$$

$$\mathbf{dV} = \mathbf{P}^\top \mathbf{dO} \quad (15)$$

$$\mathbf{dP} = \mathbf{dO} \mathbf{V}^\top \quad (16)$$

$$\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - D_i) \quad (17)$$

$$\mathbf{dQ} = \mathbf{dS} \mathbf{K} / \sqrt{d} \quad (18)$$

$$\mathbf{dK} = \mathbf{dS}^\top \mathbf{Q} / \sqrt{d}, \quad (19)$$

我们可以看到，操作序列并不需要我们在前向传播过程中存储注意力分数 $\mathbf{P}$ 在 HBM ——我们从激活值 $\mathbf{Q}$ 、 $\mathbf{K}$ 和 L 在（13）和（14）中重新计算它们。

快速注意力前向传递的细节。现在我们对FlashAttention-2所用的技术有了高层次的认识，接下来将深入探讨你将要实现的FA2前向传播核的细节。为了避免在 HBM 之间读写注意力矩阵，我们希望使用分块计算，即独立计算输出的每个分块。这要求我们能够计算 $\mathbf{P}$ 的分块，理想情况下是按查询和键的两个维度进行分块。

然而，当我们将 $\mathbf{S}$ 应用softmax时，需要将 $\mathbf{S}$ 的整行数据进行归一化以计算softmax分母，这意味着我们无法直接在瓦片中计算 $\mathbf{P}$ 。FlashAttention-2通过使用在线softmax解决了这一问题。在下文中，我们将使用下标索引 i 表示当前查询瓦片，上标索引 (j) 表示当前键瓦片。查询维度上的瓦片大小为 B_q ，键维度为 B_k 。我们不会沿隐藏维度 d 进行瓦片划分。

我们还保留了一些按行运行的值， $m_i^{(j)} \in \mathbb{R}^{B_q}$ 且 $l_i^{(j)} \in \mathbb{R}^{B_q}$ 。逐行 $m_i^{(j)}$ 值是一个运行最大值，通过追踪该值以实现数值稳定的softmax计算（回顾作业1中softmax实现的技巧）。我们将随着 j 的增加，用 $\mathbf{S}$ 的每个新行级瓦片更新 $m_i^{(j)}$ 。利用运行最大值，我们可以计算未归一化的softmax值

(分子) 作为 $\sim \mathbf{P}_i^{(j)} = \exp(\mathbf{S}_{ij} - m_i^{(j)})$ 。 $l_i^{(j)}$ 是 softmax 分母的动态代理, 将使用未归一化的 softmax 值更新为 $l_i^{(j)} = \exp(m_i^{(j-1)})$ 。 当我们最终输出时, 需要使用 $l_i^{(T_k)}$ 完成归一化, 这是处理所有关键瓦片后 $l_i^{(j)}$ 的最终值。 算法1展示了应在 GPU 上实现的前向传播过程。

算法1 FlashAttention-2前向传递

要求: $\mathbf{Q} \in \mathbb{R}^{N_q \times d}$, $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N_k \times d}$, 瓷砖尺寸 B_q, B_k 将 $\mathbf{Q}$ 划分为 $T_q = \left\lceil \frac{N_q}{B_q} \right\rceil$ 瓷砖 $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_q}$ of size $B_q \times d$

将 $\mathbf{K}, \mathbf{V}$ 划分为 $T_k = \left\lceil \frac{N_k}{B_k} \right\rceil$ 个瓷砖 $\mathbf{K}^{(1)}, \dots, \mathbf{K}^{(T_k)}$ 和 $\mathbf{V}^{(1)}, \dots, \mathbf{V}^{(T_k)}$ 的大小为 $B_k \times d$ 对于 $i = 1, \dots, T_q$

从全局内存中加载 $\mathbf{Q}_i$

初始化 $\mathbf{O}_i^{(0)} = \mathbf{0} \in \mathbb{R}^{B_q \times d}$, $l_i^{(0)} = 0 \in \mathbb{R}^{B_q}$, $m_i^{(0)} = -\infty \in \mathbb{R}^{B_q}$

对于 $j = 1, \dots, T_k$ **do**

从全局内存中加载 $\mathbf{K}^{(j)}, \mathbf{V}^{(j)}$

计算预softmax注意力分数的瓦片 $\mathbf{S}_i^{(j)} = \frac{\mathbf{Q}_i (\mathbf{K}^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$ 计算 $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_q}$

计算 $\sim \mathbf{P}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_q \times B_k}$

计算 $l_i^{(j)} = \exp(m_i^{(j-1)} - m_i^{(j)}) l_i^{(j-1)} + \text{rowsum}(\sim \mathbf{P}_i^{(j)}) \in \mathbb{R}^{B_q}$ 计算 $\mathbf{O}_i^{(j)} = \text{diag}(\exp(m_i^{(j-1)} - m_i^{(j)})) \mathbf{O}_i^{(j-1)} + \sim \mathbf{P}_i^{(j)} \mathbf{V}^{(j)}$

结束

计算 $\mathbf{O}_i = \text{diag}(l_i^{(T_k)})^{-1} \mathbf{O}_i^{(T_k)}$

计算 $L_i = m_i^{(T_k)} + \log(l_i^{(T_k)})$

将 $\mathbf{O}_i$ 写入全局内存, 作为 $\mathbf{O}$ 的第 i 个瓦片。

将 L_i 写入全局内存, 作为 L 的第 i 个瓦片。

结束

返回输出 $\mathbf{O}$ 和 $\log \text{sumexp } L$ 。

在深入探讨Triton的前向传播实现之前, 我们先整理出几个编写Triton内核时的通用技巧和实用窍门。

Triton小贴士和技巧

- 在 Triton 中, 你可以使用 `tl.device_print` 进行调试: https://triton-lang.org/main/python-api/generated/triton.language.device_print.html。有一个设置 `TRITON_INTERPRET=1`, 可以在CPU上运行Triton解释器, 但我们发现它存在bug。
- 在定义块指针时, 需确保其具有正确的偏移量, 并且块偏移量需乘以相应的瓦片尺寸。
- 螺纹块的发射网格设置为

```
kernel_fn[(launch_grid_d1, launch_grid_d2, ...)](...arguments...)
```

在 torch autograd 函数子类的方法中, 正如我们在加权求和示例中所见。

- 用 `tl` 进行矩阵乘法。点。
- 若要推进块指针，请使用 `*_block_ptr = *_block_ptr.advance (...)`

问题（闪前）：15分

- (a) 编写一个纯PyTorch（不含Triton）的自回归函数，实现FlashAttention-2的前向传播。该实现速度将显著慢于常规PyTorch版本，但有助于调试Triton内核。

你的实现应该接收输入Q、K和V以及一个标志`is_causal`，并生成输出O和对数和指数值 L 。你可以忽略这个任务中的`is_causal`标志。`autograd`函数的`forward`方法随后应使用`save L, Q, K, V, O`进行反向传播，并返回 O 。请记住`autograd`函数的`forward`方法实现总是将上下文作为其第一个参数。任何`autograd`函数类都需要实现`backward`方法，但目前你可以直接抛出`NotImplemented`。如果你需要一个对比基准，可以实现PyTorch中的公式4到6和12来比较你的输出。

随后界面`def forward (ctx, Q, K, V, is_causal=False)`。自行确定瓦片尺寸，但确保至少为 16×16 。我们始终会用2的整数次方且至少为16的维度测试代码，因此无需担心越界访问。

交付成果：一个 `torch.autograd.Function` 子类，用于在前向传播中实现 FlashAttention-2。为测试代码，请实现该子类。

`[adapters.get_flashattention_autograd_function_pytorch]`。然后，使用 `uv run pytest -k test_flash_forward_pass_pytorch` 运行测试，并确保你的实现通过该测试。

- (b) 根据算法1编写FlashAttention-2前向传播的Triton内核，然后在`torch autograd`中编写一个子类函数，该函数在前向传播时调用这个（融合）内核，而非在PyTorch中计算结果。以下是一些针对该问题的技巧：

- 为进行调试，建议将您执行的每次Triton操作结果与(a)部分中编写的PyTorch分块实现进行对比。
- 您的启动网格应设置为 $(T_q, \text{batch_size})$ ，这意味着每个Triton程序实例将仅从单个批处理索引加载元素，并且仅对Q、O和 L 中的单个查询瓦片进行读写操作。
- 该内核应仅包含一个循环，该循环将遍历键块 $1 \rightarrow j \rightarrow T_k$ 。
- 在循环末尾移动块指针。
- 使用下方的函数声明（通过我们提供的块指针，您应能推断其余指针的设置）：

```

1  @triton . 吉特
2  def flash_fwd_kernel(
3      Q_ptr, K_ptr, V_ptr,
4      O_ptr, L_ptr,
5      步长_qb、步长_qq、步长_qd
6      步长KB、步长KK、步长KD
7      步长_vb, 步长Vb, 步长_vd,
8      步长_表征, 步长_量化, 步长_优化
9      步长 lb, 步长 lq

```



```

10     查询数，键数
11     规模
12     D: 常量类型。
13     Q_tile_SIZE: tl.constexpr,
14     K_TILE_SIZE: tl.constexpr,
15 ):
16     # 程序索引
17     query_tile_index = tl.program_id(0)
18     batch_index = tl.program_id(1)
19
20     # 用对应的批处理索引对每个指针进行偏移
21     # 与每个张量的批量步长相乘
22     Q_block_ptr = tl.make_block_ptr(
23         Q指针 + 批量索引 * stride_qb
24         形状= (N_QUERIES, D)
25         步幅= (步幅_qq, 步幅_qd)
26         偏移量= (查询块索引*块大小, 0),
27         块形状= (QTile尺寸, D)
28         顺序= (1, 0), 29)
29
30     ...
31

```

其中尺度为 $\frac{1}{\sqrt{d}}$ Q_tile_SIZE和K_tile_SIZE分别是 B_q 和 B_k ，这些参数可以稍后调整。

这些附加指南可帮助您避免精密度问题：

- 片上缓冲区 ($\mathbf{O}_i$, l , m) 应具有 dtype `tl.float32`。若需将数据累加至输出缓冲区，请使用 `acc` 参数 (`acc=tl.dot(..., acc=acc)`)。
- 将 $\mathbf{P}_i^{(j)}$ 转换为 $\mathbf{V}$ 的 dtype^(j) 在乘法运算前，需将 $\mathbf{O}_i$ 转换为对应的数据类型后，再写入全局内存。转换操作通过 `tensor.to` 完成。可通过 `tensor.dtype` 获取张量的数据类型，而通过 `*_block_ptr.type.element_ty` 获取块指针/指针的数据类型。

交付成果： 一个 `torch.autograd.Function` 子类，该子类在前向传播中使用你的 Triton 内核实现 FlashAttention-2。实现

[\[adapters.get_flash_autograd_function_triton\]](#)。接着，使用 `uv run pytest -k test_flash_forward_pass_triton` 命令运行测试，确保你的实现能够通过。

- (c) 将一个标志作为最后一个参数添加到你的 `autograd` 函数中。因果掩码的函数实现。这应该是一个布尔标志，当设置为 **True** 时，启用索引比较以进行因果掩码。你的 Triton 内核应有一个对应的附加参数 `is_causal: tl.constexpr`（这是必需的类型注解）。在 Triton 中，为查询和键构建适当的索引向量，并通过比较形成大小为 $B_q \times B_k$ 的方形掩码。对于被屏蔽的元素，将常数值 `-1e6` 加到注意力分数矩阵 $\mathbf{S}_i^{(j)}$ 的对应元素上。确保保存 `backward using ctx.is_causal=is_causal` 的掩码标志。

交付物： 一个用于你的 `torch.autograd.Function` 子类的附加标志，该子类使用你的 Triton 内核实现带有因果掩码的 FlashAttention-2 前向传递。确保该标志为可选，默认为 **False**，以便之前的测试仍能通过。

实现带有重计算的反向传播 注意，与公式7至11中的标准反向传播不同，我们可以使用重计算来避免公式13至19所示反向传播中的 softmax 操作。这意味着我们可以使用一个简单的内核来计算反向传播，且无需在线技巧。因此，在此部分，你可以通过调用 `torch` 编译器在常规

问题（flash_backward）：5分

使用PyTorch（而非Triton）和torch编译器实现FlashAttention-2自适应梯度函数的反向传播。该实现应输出Q、K、V、O、dO和L张量，并返回dQ、dK和dV。请务必计算并使用D向量。可参考公式13至19的计算过程。

可交付成果：为测试您的实现，请运行 `uv run pytest -k test_flash_backward`。

现在，让我们比较一下您（部分）基于Triton的FlashAttention-2实现与基于PyTorch的常规注意力机制实现的性能表现。

问题（flash_benchmarking）：5分

- (a) 使用 `triton.testing.do_bench` 编写一个基准测试脚本，比较你（部分）Triton实现的FlashAttention-2前向和后向传递与常规PyTorch实现（即不使用FlashAttention）的性能。

具体来说，您需要生成一个包含前向、反向及端到端双向传输延迟的表格，分别对应Triton和PyTorch两种实现方案。在开始基准测试前，需随机生成所有必要输入数据，并在单个H100节点上运行测试。实验时务必使用批量大小1和因果掩码技术。测试参数需覆盖以下组合：序列长度（从128到65536，按2的幂次递增）、嵌入维度（从16到128，按2的幂次递增）以及 `torch.bfloat16` 和 `torch.float32` 两种精度类型。根据输入数据规模的不同，可能需要相应调整数据分块的大小。

可交付成果：一份结果对比表，用于比较您对FlashAttention-2的实现与PyTorch实现的差异，采用上述设置，并报告前向延迟、后向延迟及端到端延迟。

1.3.3 FlashAttention-2排行榜

作业2的排行榜将测试你对FlashAttention-2的实现速度（包括两个方面（前传与回传）。我们挑战您进一步提升设备性能

使用你能想到的任何技巧。限制条件是你不能更改输入/输出。

关于该函数，必须使用Triton（遗憾的是不支持CUDA）。您的输入将在BF16架构下进行测试。

因果掩蔽，且必须通过与常规实现相同的测试。该实现必须

也应由你自己掌控，且不得使用现成的实现方案。你的时机应以...为基准来衡量

H100，样本批处理大小为1，查询、键和值的序列长度为16,384， $d_{\text{model}} = 1024$

包含16个头部。我们将验证前5-10个提交的正确性和性能。我们将进行测试

您的实施方案的截止时间如下：

```
1 def test_timing_flash_forward_backward():
2     n_head = 16
3     d_head = 64
4     seq_length = 16384
5     q, k, v = torch.randn(
6         3, n_head, sequence_length, d_head, device='cuda', dtype=torch.bfloat16, requires_grad=True
7     )
8
```

```
‣flash = torch.compile (FlashAttention2.apply)
```

```

10
11 def flash_forward_backward () :
12     o = flash (q, k, v, True)
13     损失 = o .sum ()
14     损失 .backward ()
15
16 个结果=triton.测试.do_bench (向前向后, rep=10000, warmup=1000)
17 打印 (结果)

```

为便于测试，可将重复次数和预热时间（以毫秒计）缩短至更短数值。改进建议如下：

- 调整内核的磁贴大小（使用Triton自动调优功能！）
- 调整Triton配置参数
- 在Triton中实现反向传递，而不仅仅是火炬编译（参见下文1.3.4节）。
- 在反向传递过程中，需对输入数据进行两次处理：一次针对dQ，另一次针对dK和dV，以避免原子操作或模块间同步问题。
- 在进行因果掩蔽时提前终止程序实例，跳过所有始终为零的瓦片
- 将未掩码的瓦片与瓦片对角线分离，计算第一种方法无需比较索引，第二种方法仅需一次比较
- 在H100上使用 TMA（Tensor Memory Accelerator）功能，遵循与本教程相似的模式。

将 你 的 最 佳 成 绩 提 交 到 github.com/stanford-cs336/assignment2-systems-leaderboard排行榜

1.3.4 可选：Triton反向传递

若您希望在Triton上获得更多实践机会或快速提交排行榜数据，我们提供了下方的FlashAttention-2反向传播模块，您可在Triton中直接实现。算法2展示了该模块在Triton中的具体实现方式。其中关键技巧在于对P进行两次计算：一次用于dQ的反向传播，另一次用于dK和dV。这种设计可避免跨线程块的同步操作。

算法2 Tiled FlashAttention-2反向传播

要求: $Q, O, dO \in \mathbb{R}^{N_q \times d}$, $K, V \in \mathbb{R}^{N_k \times d}$, $L \in \mathbb{R}^{N_q}$, 瓷片尺寸 B_q, B_k 计算 $D = \text{rowsum}(dO \circ O) \in \mathbb{R}^{N_q}$

将 Q, O, dO 分割为 $T_q = \left\lceil \frac{N_q}{B_q} \right\rceil$ 瓷片 $Q_1, \dots, Q_{T_q}, O_1, \dots, O_{T_q}, dO_1, \dots, dO_{T_q}$, 每个尺寸为 $B_q \times d$

将 K, V 划分为 $T_k = \left\lceil \frac{N_k}{B_k} \right\rceil$ 瓷片 $K^{(1)}, \dots, K^{(T_k)}$ 与 $V^{(1)}, \dots, V^{(T_k)}$, 每个尺寸为 $B_k \times d$

将 L, D 分割为 T_q 个瓦片 $L_1, \dots, L_{T_q}$ 和 $D_1, \dots, D_{T_q}$, 每个瓦片的大小为 B_q 对于 $j = 1, \dots, T_k$ **do**

 负荷 $K^{(j)}, V^{(j)}$ 来自全局存储器

 初始化 $dK^{(j)} = dV^{(j)} = \mathbf{0} \in \mathbb{R}^{B_k \times d}$

对于 $i = 1, \dots, T_q$

 从全局内存中加载 Q_i, O_i, dO_i, dQ_i

 计算注意力分数的瓦片 $S_i^{(j)} = \frac{Q_i (K^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$

 计算注意力概率 $P_i^{(j)} = \exp(S_i^{(j)} - L_i) \in \mathbb{R}^{B_q \times B_k}$

 计算 $dV_i^{(j)} = (P_i^{(j)})^\top dO_i \in \mathbb{R}^{B_k \times d}$

 计算 $dP_i^{(j)} = dO_i V_i^{(j)} \in \mathbb{R}^{B_q \times B_k}$

 计算 $dS_i^{(j)} = P_i^{(j)} \circ (dP_i^{(j)} - D_i) / \tilde{d} \in \mathbb{R}^{B_q \times B_k}$

从全局内存加载 dQ_i , 然后更新 $dQ_i += dS_i^{(j)} K^{(j)} \in \mathbb{R}^{B_q \times d}$, 并写回全局内存。必须是原子操作才能保证正确性!

 计算 $dK^{(j)} += (dS_i^{(j)})^\top Q_i \in \mathbb{R}^{B_k \times d}$ 。

结束

 将 $dK^{(j)}$ 和 $dV^{(j)}$ 写入全局内存, 作为 dK 和 dV 的第 j 个块。

结束

返回 dQ, dK, dV 。

2 分布式数据并行训练

在本任务的后续部分，我们将重点探讨如何利用多GPU进行语言模型训练，重点关注数据并行性。首先，我们将从PyTorch的分布式通信基础讲起。接着，我们会研究分布式数据并行训练的简易实现方案，随后通过实际测试验证多种提升通信效率的优化方案。

2.1 PyTorch中的单节点分布式通信

我们先从一个简单的PyTorch分布式应用开始，目标是生成四个随机整数张量并计算它们的总和。

在下面的分布式情况下，我们将生成四个工作进程，每个进程生成一个随机整数张量。为了对这些张量进行求和，我们将调用**all-reduce**集体通信操作，该操作将每个进程上的原始数据张量替换为全部归并的结果（即求和）。

现在让我们看看一些代码。

```
1 导入os
2 导入torch
3 导入torch。分布式作为dist
4 导入torch.multiprocessing作为mp
5
6 def setup (rank, world_size):
7     os.environ[ "MASTER_ADDR" ] = "localhost"
8     操作系统 o 约[ "MASTER_PORT" ] = "29500"
9     远程初始化进程组 ( "gloo" , rank=rank, world_size=world_size)
10
11 def distributed_demo (rank, world_size):
12     设置 (等级, 世界大小)
13     数据 = torch.randint (0, 10, (3, ))
14     打印 (f "rank {rank} 数据 (在全归约前): {数据}" )
15     dist.all_reduce (数据, 异步操作=False)
16     打印 (f "rank {rank} 数据 (全归约后): {数据}" ) 17
18 if __name__ == "__main__":
19     世界大小 =4
20 mp.spawn (fn=distributed_demo, args=(world_size, ), nprocs=world_size, join=True)
```

运行上述脚本后，我们得到如下输出。正如预期的那样，每个工作进程最初都持有不同的数据张量。在所有归约操作之后，该操作对所有工作进程中的张量进行求和，数据在每个工作进程上原地修改，以持有所有归约结果。²

```
1 $ 用Python运行分布式Hello_world.py
2 秩3数据 (全归约前): 张量 ([3,7,8])
3 秩0数据 (全归约前): 张量 ([4,4,7])
4 秩2数据 (全归约前): 张量 ([6,0,7])
5 秩1数据 (全归约前): 张量 ([9,5,3])
6 秩1数据 (全归约后): 张量 ([22,16,25])
7 秩0数据 (全归约后): 张量 ([22,16,25])
```

² 若多次运行该脚本，会发现打印输出的顺序并非确定性的。由于该应用程序运行在分布式环境中，我们无法控制命令执行的具体顺序——唯一能保证的是：当全归约操作完成后，各独立进程将持有完全相同的位结果张量。

8秩3数据（全归约后）：张量（[22,16,25]）

9秩2数据（全归约后）：张量（[22,16,25]）

现在让我们更仔细地回顾上面的脚本。命令`mp.spawn`会生成`nprocs`个进程，这些进程会运行指定参数的`fn`函数。此外，`fn`函数的调用格式为`fn(rank, *args)`，其中`rank`表示工作进程的索引（取值范围为0到`nprocs-1`）。因此，我们的`distributed_demo`函数必须将这个整数`rank`作为第一个位置参数。同时，我们还需要传递`world_size`参数，该参数表示工作进程的总数。

这些工作进程均隶属于一个**进程组**，该进程组通过`dist.init_process_group`函数进行初始化。进程组由多个工作进程组成，这些进程通过共享主节点进行协调与通信。主节点由其IP地址和端口定义，并以0号进程身份运行。诸如全归约（all-reduce）等集体通信操作将作用于进程组中的每个进程。

本案例中，我们使用“gloo”后端初始化了进程组，但其他后端也可供选择。具体而言，“nccl”后端将采用NVIDIA NCCL集体通信库，该库通常对CUDA张量的性能表现更优。但需注意，NCCL仅支持GPU架构，而Gloo则可在纯CPU机器上运行。经验法则是：分布式GPU训练使用NCCL，分布式CPU训练和/或本地开发使用Gloo。本例选用Gloo是因为它支持纯CPU机器的本地执行与开发。

在运行多GPU任务时，确保不同`rank`使用不同的GPU。一种实现方法是在`setup`函数中调用`torch.cuda.set_device(rank)`，使得`tensor.to("cuda")`会自动将其移动到指定设备。另一种方法是显式创建每个`rank`的设备字符串（例如`device=f"cuda:{rank}"`），然后将此设备字符串作为任何数据移动的目标设备（例如`tensor.to(f"cuda:{rank}")`）。

术语。在本作业的其余部分（以及您可能在网上看到的各种其他资源中），您可能会在PyTorch分布式通信的上下文中遇到以下术语。虽然我们将重点讨论单节点、多进程的分布式训练，但这些术语对于理解分布式训练的总体情况很有帮助。请参见图2的可视化表示。

节点：网络上的机器。

worker：参与分布式训练的程序实例。在本任务中，每个worker将拥有一个单独的进程，因此我们将`worker`、`process`和`worker process`互换使用。但需要注意的是，一个worker可能使用多个进程（例如用于加载训练数据），因此这些术语在实际应用中并不总是等同。

世界大小：进程组中总工作数。

全局排名：一个整数ID（0到`world_size-1`之间），用于唯一标识进程组中的工作进程。例如，当`world_size`为2时，一个进程的全局排名为0（主进程），另一个进程的排名为1。

本地世界大小：在跨节点运行应用程序时，全局工作量（`world size`）指特定节点上本地运行的工作者数量。例如，若某应用程序在2个节点上各分配4个工作者，则全局工作量为8，本地工作量为4。需注意，当程序在单个节点运行时，工作者的本地工作量即等同于全局工作量。

本地排名：一个整数ID（范围在0到`local_world_size-1`之间），用于唯一标识机器上本地工作进程的索引。例如，假设某个应用程序在2个节点上各生成4个进程，那么每个节点上的工作进程本地排名将依次为0、1、2和3。需要注意的是，在运行单节点多进程分布式应用程序时，进程的本地排名与其全局排名具有等效性。

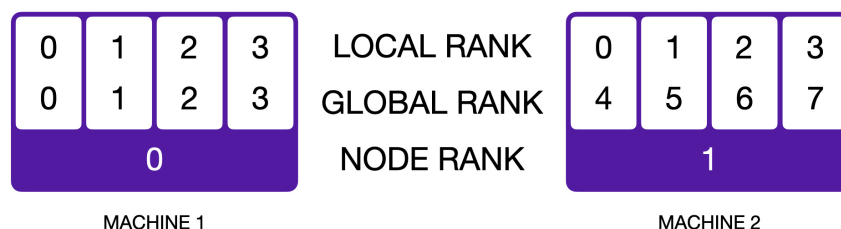


图2:一个在2个节点上运行的分布式应用程序的示意图，世界大小为8。每个工作进程由全局排名（从0到7）和本地排名（从0到3）标识。图取自lightning.ai/docs/fabric/stable/advanced/distributed_communication.html

2.1.1 分布式应用基准测试的最佳实践

在本任务的这一部分，您将通过基准测试分布式应用程序，以更深入地理解通信开销。以下是一些最佳实践：

- 在条件允许时，于同一台机器上运行基准测试以实现受控比较。
- 在计时目标操作前需执行若干预热步骤。这对于NCCL通信呼叫尤为重要。通常5次预热迭代即可满足需求。
- 调用 `torch.cuda.synchronize()` 在 GPU 上进行基准测试时等待 CUDA 操作完成。请注意，即使调用通信操作时使用 `async_op=False`，这也是必要的，因为该操作是在 GPU 上排队时返回的（而不是在通信实际完成时）。³
- 不同等级的计时可能略有差异，因此通常需要跨等级汇总测量数据以提高估计精度。您可能会发现全局收集的集合（具体为`dist.all_gather_v`对象函数）对于收集所有等级的结果非常有用。
- 通常情况下，先在CPU端使用Gloo进行本地调试，随后根据具体问题需求，再在GPU端使用NCCL进行基准测试。在不同后端之间切换仅需修改`init_process_group`调用及张量设备类型转换即可。

问题（分布式通信单节点）：5分

编写脚本对单节点多进程架构中的全归约操作运行时进行基准测试。上文示例代码可作为合理起点。请尝试调整以下参数设置：

后台+设备类型： Gloo + CPU，NCCL + GPU.

全归约数据量： 浮点32位数据张量，覆盖1MB、10MB、100MB、1GB。

进程数： 2、4或6个进程。

资源需求： 最多6个GPU。每次基准测试运行时间应少于5分钟。

³ 参见github.com/pytorch/pytorch/issues/68112#issuecomment-965932386 了解更多细节。

交付物：比较不同设置的图表和/或表格，并附2-3句话的评论，说明您的结果以及对各种因素相互作用的看法。

2.2 分布式数据并行训练的朴素实现

既然我们已经掌握了用PyTorch编写分布式应用的基础知识，现在就来构建一个分布式数据并行（DDP）训练的简易实现。

数据并行处理技术将数据批次分配至多个设备（如GPU），从而支持对单个设备无法承载的大型批次进行训练。例如，若四台设备每台最大可处理32个批次，采用数据并行训练可实现128个批次的有效处理。

以下是分布式数据并行训练的简易步骤：首先，每个设备会构建一个随机初始化的模型。我们通过广播式集体通信操作，将模型参数从第0号节点传输至所有其他节点。训练开始时，各设备均持有完全相同的模型参数和优化器状态副本（例如Adam优化器中累积的梯度统计信息）。

1. 给定一个包含 n 个样本的批次，该批次会被分片，每个设备接收 n/d 个不相交的样本（其中 d 是用于数据并行训练的设备数量）。 n 应能整除 d ，因为训练时间受最慢进程的限制。
2. 每个设备使用其本地的模型参数副本，对它的 n/d 个示例进行前向传递，并通过反向传递来计算梯度。请注意，在此阶段，每个设备都持有从其接收的 n/d 个示例计算出的梯度。
3. 然后我们使用全归约的集体通信操作来平均不同设备上的梯度，因此每个设备都持有所有 n 个样本的平均梯度。
4. 随后，每个设备运行优化器步骤以更新其参数副本——从优化器的角度来看，这仅仅是优化一个本地模型。由于所有设备均基于相同的初始模型和优化器状态运行，并在每次迭代中采用相同的平均梯度，因此参数与优化器状态将在所有设备间保持同步。至此，我们已完成单次训练迭代，可重复该流程。

问题（naive_ddp）：5分

交付成果：编写脚本，通过在反向传播后对所有参数梯度进行全归约，实现分布式数据并行训练。为验证 DDP 实现的正确性，请使用该脚本在随机生成的数据上训练小型玩具模型，并验证其权重是否正确。

与单过程训练结果相匹配。



a 如果你在编写这个测试时遇到困难，可能可以参考 `tests/test_ddp_individual_parameters.py`

问题（naive_ddp_benchmarking）：3分

在这一朴素的 DDP 实现中，每次反向传播后参数都会按秩逐一进行全降维。为更直观地评估数据并行训练的开销，建议编写脚本，用这种朴素的 DDP 实现对先前实现的语言模型进行基准测试。测量每个训练步骤的总耗时及梯度通信所占时间比例。在单节点配置（1个节点×2个GPU）下，针对1.1.2节所述XL模型规模进行测量。

交付物：基准测试设置的描述，以及每次训练的测量时间

每次设置的梯度迭代次数及沟通时间。

2.3 改进最小 DDP 实现

我们在第2.2节看到的最小 DDP 实现存在几个关键限制：

1. 该算法对每个参数张量执行独立的全归约操作。每次通信调用都会产生开销，因此将通信调用分批处理以最小化这种开销可能是有利的。
2. 该机制需待反向传播完成后再进行梯度传输。但由于反向传播采用增量计算方式，当某个参数的梯度计算完毕后，即可立即传输，无需等待其他参数的梯度结果。这种设计实现了梯度传输与反向传播计算的同步化，有效降低了分布式数据并行训练的计算开销。

在本部分任务中，我们将逐一分析这些限制因素，并评估其对训练速度的影响。

2.3.1 减少通信呼叫次数

与其为每个参数张量分别发出通信调用，不如尝试通过批量执行全归约操作来提升性能。具体而言，我们将对需要全归约的梯度进行处理：先将它们拼接成单一张量，再对所有秩的组合梯度执行全归约操作。此时可借助`torch.utils.flatten_dense_tensors`和`unflatten_dense_tensors`这两个函数来实现。

问题（最小ddp平坦基准测试）：2分

修改最小 DDP 实现以传递包含所有参数梯度的张量。在先前使用条件下（1个节点×2个GPU，如1.1.2节所述的XL模型大小），将其性能与最小 DDP 实现进行对比，后者对每个参数张量执行全归约操作。

可交付成果：在分布式数据并行训练中，通过单次批量全归约调用，测量每次训练迭代的耗时及梯度通信时间。用1-2句话对比批量处理与逐个通信梯度的结果。

2.3.2 参数梯度的重叠计算与通信

虽然批量处理通信调用可能有助于降低执行大量小型全归约操作时产生的开销，但通信时间本身仍会直接增加系统开销。为解决这一问题，我们可以利用反向传播过程的特性——该过程会逐层（从损失函数开始向输入端推进）增量计算梯度。因此，当参数梯度计算完成后，我们就能立即进行全归约操作，通过将反向传播的计算与梯度传输过程进行时间重叠，从而有效降低数据并行训练的开销。

我们将首先实现并测试一个分布式数据并行封装器，该封装器在反向传播过程中异步对各参数张量进行全归约处理。以下指针可能具有参考价值：

后向钩子若要在参数的梯度在反向传播中被累积后自动调用函数，可以使用 `register_post_accumulate_grad_hook` 函数。⁴

⁴更多信息和使用示例，请参见pytorch.org/docs/stable/generated/torch.Tensor.register_post_accumulate_grad_hook.html。

异步通信所有PyTorch的集体通信操作都支持同步执行（`async_op=False`）和异步执行（`async_op=True`）。同步调用会阻塞等待，直到集体操作被排队到GPU上。但这并不意味着 CUDA 操作已经完成，因为 CUDA 操作本身是异步的。不过，后续使用输出结果的函数调用仍会按预期运行。⁵相比之下，异步调用会返回分布式请求句柄——这意味着当函数返回时，集体通信操作可能尚未被排队到GPU上，更不用说完成。若需等待操作被排队到GPU（从而确保输出结果可用于后续操作），可调用返回的通信句柄上的`handle.wait()`函数。

例如，以下两个示例均通过同步或异步调用，对张量列表中的每个张量执行全归约操作：

```
1 张量 = [torch.rand(5) for _ in range(10)]
2
3  # 同步操作，直至GPU上操作排队。
4  for tensor in tensors:
5      英里。all_reduce(tensor, async_op=False)
6
7  # 异步调用，每次调用后立即返回
8  # 等待最后的结果。
9  个处理项 = []
10 for tensor in tensors:
11     handle = dist.all_reduce(tensor, async_op=True)
12     处理.append(handle)
13
14     # ...
15     # 可能执行不依赖于all_reduce结果的其他命令 16# ...
17
18     确保所有-reduce调用均已排队
19     因此，其他依赖于
20     # all-reduce 输出可以排队。
21 用于处理在 处理项中：
22 handle.wait()
23 处理.clear()
```

问题（`ddp_overlap_individual_parameters`）：5分

需实现一个Python类来处理分布式数据并行训练。该类应封装任意PyTorch神经网络模块（`nn.Module`），负责训练前权重的广播操作（确保所有节点初始参数一致），并调用梯度平均通信调用。建议采用以下公共接口：

`def init(self, module: torch.nn.模块)`：给定一个实例化的PyTorch `nn.模块`需要并行化，构建一个DDP容器来处理跨rank的梯度同步。

`def forward(self, *inputs, **kwargs)`：Calls the wrapped module's `forward()` method with the provided positional and keyword arguments.

`def finish_gradient_synchronization(self)`：调用时，等待异步通信

⁵在高级案例中，若使用多个CUDA流，可能需要跨流显式同步，以确保输出可为后续操作做好准备。详见pytorch.org/docs/stable/notes/cuda.html#cuda-streams。

需在GPU上排队等待

为使用该类进行分布式训练，需先传递待封装的模块，随后在调用优化器步骤前添加对`finish_gradient_synchronization()`的调用，以确保依赖梯度的优化器步骤能够被排队执行。

```
model = ToyModel().to(device)
ddp_model = DDP(model)
```

对于在范围（train_steps）内：

```
x, y = get_batch()
logits = ddp_model(x)
loss = loss_fn(logits, y)
loss.backward()
ddp_model.finish_gradient_synchronization()
optimizer.step()
```

交付成果：实现一个容器类来处理分布式数据并行训练。该类应覆盖梯度通信和反向传播计算。为测试您的 DDP 类，首先实现适配器`[adapters.get_ddp_individual_parameters]`和`[adapters.ddp_individual_parameters_on_after_backward]`（后者是可选的，根据您的实现可能不需要它）。

随后，为执行测试，请运行 `uv run pytest tests/test_ddp_individual_parameters.py`。建议多次运行测试（例如5次），以确保其可靠通过。

问题（ddp_overlap_individual_parameters_benchmarking）：1 分

- (a) 在将反向传播计算与单个参数梯度的通信相结合时，对 DDP 实现的性能进行基准测试。将其性能与我们先前研究的设置（最小 DDP 实现，即对每个参数张量执行全归约，或对所有参数张量的拼接执行单次全归约）进行对比，设置相同：1个节点、2个GPU，以及1.1.2节所述的XL模型大小。

可交付成果：在将反向传播与单个参数梯度通信重叠时，每次训练迭代的测量时间，并附1-2句话对比结果。

- (b) 使用Nsight性能分析器对基准测试代码（配置为1个节点、2个GPU、XL模型大小）进行分析，对比初始 DDP 实现与本 DDP 实现（后者将计算与通信重叠）。通过可视化对比两组性能轨迹，并提供分析器截图证明：一种实现方式将计算与通信重叠，而另一种则未实现这种重叠。

交付成果：2张截图（一张来自初始 DDP 实现，另一张来自本 DDP 实现，该实现将计算与通信重叠），直观展示通信是否与反向传播存在重叠。

2.3.3 分桶参数梯度的重叠计算与通信

在前一节（§2.3.2）中，我们将反向传播计算与单个参数梯度的通信进行了叠加。然而我们发现，批量通信调用能显著提升性能，尤其在处理大量参数张量时（这在深度Transformer模型中很常见）。我们之前的批量处理尝试是将所有梯度一次性发送，这需要等待反向

为实现高效完成，本节将采用双管齐下的策略：首先将参数划分为多个桶（从而减少总通信调用次数），当各构成张量准备就绪时，再将这些桶统一归并（实现通信与计算的协同处理）。

问题（ddp_overlap_bucketed）：8分

开发一个Python类实现分布式数据并行训练，采用梯度分桶技术提升通信效率。该类需封装任意PyTorch神经网络模块，负责训练前权重广播（确保所有节点初始参数一致）以及梯度平均的分桶通信调用。推荐采用以下接口：

def init(self, module: torch.nn.Module, bucket_size_mb: float): 给定一个需要并行化的PyTorch nn.Module实例，构建一个 DDP 容器以处理跨rank的梯度同步。梯度同步应采用分桶存储，每个桶最多容纳bucket_size_mb个参数。

def forward(self, *inputs, **kwargs): Calls the wrapped module's forward() method with the provided positional and keyword arguments.

def finish_gradient_synchronization(self):调用时，等待异步通信调用在GPU上排队。

除了新增bucket_size_mb初始化参数外，该公共接口与我们先前的 DDP 实现保持一致，即每个参数单独传递。我们建议采用模型参数（）的逆序来分配参数至桶，因为在反向传播过程中梯度将按此顺序生成。

可交付成果：实现一个容器类以处理分布式数据并行训练。该类应覆盖梯度通信和反向传播的计算。梯度通信应采用分桶方式，以减少总通信调用次数。为测试您的实现，请完成[adapters.get_ddp_bucketed]、[adapters.ddp_bucketed_on_after_backward] 以及 [adapters.ddp_bucketed_on_train_batch_start]（后两个参数为可选配置，根据具体实现方案可能无需启用）。

随后，为执行测试，请运行 `pytest tests/test_ddp.py`。建议多次运行测试（例如5次），以确保其可靠通过。

问题（ddp_bucketed_benchmarking）：3分

- (a) 使用与先前实验相同的配置（1个节点、2个GPU、XL模型大小）对分桶 DDP 实现进行基准测试，调整最大分桶大小（1、10、100、1000MB）。将测试结果与未使用分桶的实验数据对比——结果是否符合预期？若存在偏差需分析原因。必要时可借助PyTorch性能分析器来深入理解通信调用的执行顺序。若需获得符合预期的结果，建议对实验设置进行哪些调整？

可交付成果：不同桶大小下每次训练迭代的测量时间。包含3-4句话的评论，说明结果、您的预期以及任何不匹配的潜在原因。

- (b) 假设计算单个梯度桶所需的时间与传输梯度桶所需的时间相同。建立一个方程来建模 DDP 的通信开销（即反向传播后额外耗费的时间），将其表示为一个函数。

模型参数总大小（字节）（ s ）、全归约算法带宽（ w ，计算方式为每个秩数据的大小除以完成全归约所需的时间）、每次通信调用相关的开销（秒）（ o ）以及桶的数量（ nb ）。

根据该方程，推导出最小化 DDP 开销的最优桶容量方程。**交付成果：**包含 DDP 开销建模方程及最优桶容量方程。

2.4 四维并行性

尽管实现上更为复杂，但我们可以沿更多维度对训练过程进行并行化。最常见的是讨论5种并行化方法：

- **数据并行计算（DP）**——将数据批次分配至多个设备，每个设备计算其对应批次的梯度。这些梯度需通过某种方式在设备间进行平均。
- **完全分片数据并行（FSDP）**——优化器状态、梯度和权重被分散到多个设备上。若仅使用动态规划（DP）和 FSDP，每个设备都需先收集所有其他设备的权重片段，才能执行前向或反向传播。
- **张量并行（TP）**——激活值被划分到新维度，每个设备计算其对应分片的输出结果。通过张量并行，我们可以将操作划分到输入或输出维度。若将权重和激活值按对应维度划分，张量并行可与 FSDP 有效结合使用。
- **管道并行性（PP）**——该模型按层级分割为多个阶段，每个阶段在不同的设备上运行。
- **专家并行计算（EP）**——在专家混合模型中，我们将专家分配至不同设备，各设备分别计算其对应专家的输出结果。

通常，我们总是将 FSDP 和 TP 合并，因此可将它们视为单一并行轴。这样就形成了4个并行轴：DP、FSDP / TP、PP 和 EP。我们还将聚焦于密集模型（而非多模态模型），因此不再深入讨论 EP。

在讨论分布式训练时，我们通常将集群描述为设备的**网格**，其中网格的轴线是我们定义并行性的方向。例如，如果有16个GPU，而模型的规模远超单个设备的承载能力，我们可能会倾向于将网格组织成 4×4 的GPU网格，其中第一维代表数据并行性（DP），第二维代表组合的 FSDP 和 TP。

参见《TPU 规模化手册》第5部分的概述（Austin等人）[2025]）有关这些方法如何工作以及如何推导其通信和内存开销的更多细节（这对于解决下面的问题将特别有帮助）。关于流水线并行的更详细讨论，请参阅超大规模操作手册附录（Nouamane Tazi[2025]）。本书其余部分也包含许多其他可能有用的信息。

问题（通信会计）：10分

考虑一个新模型配置 XXL，其中 $d_{\text{model}}=16384$ ， $d_{\text{ff}}=53248$ ， $\text{num_blocks}=126$ 。由于超大规模模型中绝大多数浮点运算（FLOPs）集中在前馈网络，我们采用以下简化假设：首先省略注意力机制、输入嵌入层和输出线性层；其次假设每个 FFN 仅包含两个线性层（忽略激活函数），其中第一层输入维度为 d_{model} 、输出维度为 d_{ff} ，第二层输入维度为 d_{ff} 、输出维度为 d_{model} 。模型由 num_blocks 个此类线性层模块构成。无需进行激活函数检查点操作，保持激活值和梯度通信采用 BF16 浮点类型，而累积梯度、主权重及优化器状态则应采用 FP32 浮点类型。

- (a) 若需在单个设备上以FP32精度存储主模型权重、累积梯度和优化器状态，需要多少内存？反向传播所需的内存（这些数据将以BF16精度存储）又占用了多少？这相当于多少块H100 80GB显存的容量？

交付物：你的计算和一句话的回复。

- (b) 现在假设你的主权重、优化器状态、梯度和一半的激活值（实际操作中每第二层）分布在 N_{FSDP} 个设备上。请写出每个设备需要多少内存的表达式。要使总内存成本低于1v5p TPU（每个设备95GB）， N_{FSDP} 需要取什么值？**交付成果：**你的计算结果和一句话回答。

- (c) 仅考虑前向传递。使用 $W_{\text{ici}} = 2 \cdot 9 \cdot 10^{10}$ 的通信带宽和 $C = 4.6 \cdot 10^{14}$ 的浮点运算速度（TPU v5p），如《TPU 扩展手册》所述。遵循扩展手册的符号约定，使用 $M_X = 2$ 、 $M_Y = 1$ （三维网格），其中 $X = 16$ 为FSDP维度， $Y = 4$ 为TP维度。该模型在何种单设备批量大小下达到计算瓶颈？在此设置下整体批量大小是多少？

交付物：你的计算和一句话的回复。

- (d) 在实际应用中，我们希望模型的批量处理规模尽可能小，同时确保计算资源的高效利用（即避免因通信开销受限）。那么，在保持高吞吐量的前提下，还能采取哪些策略来进一步缩减模型的批量处理规模呢？

交付成果：一段式回答。用参考文献和/或公式支持你的主张。

3 优化器状态分片

分布式数据并行训练在概念上简单且通常非常有效，但要求每个节点都持有模型参数和优化器状态的独立副本。这种冗余性会带来显著的内存开销。例如，AdamW优化器为每个参数维护两个浮点数，这意味着它消耗的内存是模型权重的两倍。Rajbhandari等人[2020] 通过将(1)优化器状态、(2)梯度和(3)参数在不同节点间进行分区，并根据需要在工作节点间进行通信，本文描述了多种减少数据并行训练中这种冗余的方法。

在本任务模块中，我们将通过简化优化器状态分片机制来降低各rank的内存占用。具体而言，每个rank的优化器实例仅处理部分参数（约占world_size的1/10），而非存储所有参数的优化器状态。当优化器执行优化步骤时，仅会更新其分片中的模型参数子集。随后，每个等级将向其他等级广播其更新后的参数，以确保模型参数在每次优化器步骤后保持同步。

问题（优化器状态分片）：15分

实现一个Python类以处理优化器状态分片。该类应封装任意输入的PyTorch优化器，并在每次优化器步骤后同步更新参数。我们建议采用以下公共接口：

def init (self, params, optimizer_cls: Type[Optimizer], **kwargs: Any) :初始化分片状态优化器。params 是待优化的参数集合（或参数组，若用户希望为模型不同部分使用不同的超参数，如学习率）；这些参数将在所有rank上进行分片。optimizer_cls 参数指定要封装的优化器类型（例如，optim.AdamW）。最后，任何剩余的键参数将转发至 optimizer_cls 的构造函数。请确保在此方法中调用 torch.optim.Optimizer 超类构造函数。

def step(self, closure, **kwargs): Calls the wrapped optimizer's step() method with the provided closure and keyword arguments. After updating the parameters, synchronize with the other ranks.

def add_param_group (self, param_group:dict[str, Any]) :此方法应在分片优化器中添加参数组。该方法由超类构造函数在构建分片优化器时调用，也可在训练期间调用（例如用于模型中逐层解冻）。因此，此方法需负责在各层级间分配模型参数。

交付成果：实现一个容器类来处理优化器状态分片。要测试你的分片优化器，请先实现适配器 `[adapters.get_shardedoptimizer]`。然后，要执行测试，请运行 `uv run pytest tests/test_shardedoptimizer.py`。我们建议多次运行测试（例如5次），以确保其可靠通过。

在完成优化器状态分片的实现后，我们将重点分析其对训练阶段峰值内存占用的影响，以及运行时的额外开销。

问题（优化器状态共享计费）：5分

- (a) 编写脚本分析语言模型训练过程中峰值内存使用情况，对比优化器状态分片与非分片两种配置。实验采用标准配置（1个节点、2个GPU、XL模型规模）。

分别在模型初始化后、优化器步骤执行前及执行后记录峰值内存使用量。测试结果是否符合预期？需详细分解各参数配置下的内存使用情况（例如参数占用内存、优化器状态占用内存等）。

可交付成果：2-3句话的回复，包含峰值内存使用结果，并详细说明内存如何在不同模型和优化器组件之间分配。

- (b) 我们的优化器状态分片实现如何影响训练速度？测量标准配置（1个节点、2个GPU、XL模型大小）下每次迭代的时间消耗，分别对比优化器状态分片与非分片情况。

交付物：用2-3句话回答并说明时间。

- (c) 我们的优化器状态分片方法与ZeRO第一阶段（在Rajbhandari等人2020中描述为ZeRODP P_{os} ）有何不同？

交付成果：用2-3句话总结任何差异，特别是与记忆和沟通量相关的差异。

4 尾声

恭喜你完成任务！希望你觉得这个任务既有趣又有收获，还学到了如何通过提升单GPU速度和/或利用多GPU来加速语言模型训练。

参考文献

Tri Dao. Flashattention-2: 具有更好并行性和工作分区的更快注意力机制, 2023年。URL <https://arxiv.org/abs/2307.08691>。

Tri Dao、Daniel Y Fu、Stefano Ermon、Atri Rudra 和 Christopher Re。Flashattention: 具有 IO 意识的快速且内存高效的精确注意力机制。收录于 Alice H. Oh、Alek Agarwal、Danielle Belgrave 和 Kyunghyun Cho 主编的《神经信息处理系统进展》(*Advances in Neural Information Processing Systems*), 2022 年。网址 <https://openreview.net/forum?id=H4DqfPSibmx>。

马克西姆·米拉科夫与娜塔莉亚·吉梅尔谢因。softmax函数的在线归一化计算, 2018年。网址<https://arxiv.org/abs/1805.02867>。

Horace He。从第一原理出发, 让深度学习变得令人兴奋。2022年。URL https://horace.io/brrr_intro.html。

Jacob Austin、Sholto Douglas、Roy Frostig、Anselm Levskaya、Charlie Chen、Sharad Vikram、Federico Lebron、Peter Choy、Vinay Ramasesh、Albert Webson和Reiner Pope。如何扩展你的模型。2025年。检索自<https://jax-ml.github.io/scaling-book/>。

赵浩军、阮福、穆罕默德·梅库里、莱安德罗·韦拉、托马斯·沃尔夫、努阿曼·塔齐、费迪南德·莫姆。超大规模训练手册: 基于GPU集群的训练模型, 2025年。

萨米扬·拉吉班达里、杰夫·拉斯利、奥拉通吉·鲁瓦塞和何玉雄。ZeRO: 面向万亿参数模型训练的内存优化, 2020年。arXiv:1910.02054。